

Exploration des concepts et des techniques
d'apprentissage par renforcement : De la théorie à la
pratique

TRAVAIL DE MASTER RÉALISÉ EN VUE DE
L'OBTENTION DU MASTER OF SCIENCE HES-SO EN
SCIENCES DE L'INFORMATION PAR :

DAVID PAULINO

CONSEILLERS AU TRAVAIL DE MASTER :

PR ALEXANDROS KALOUSIS

GENÈVE, LE 15 JUIN 2026

Haute école de Gestion de Genève (HEG-GE)

Filière Information Science

1 Déclaration

Ce travail de Master est réalisé dans le cadre du Master en Sciences de l'information de la Haute école de gestion de Genève.

L'étudiant-e atteste que le travail rendu est le fruit de sa réflexion personnelle, a été rédigé de manière autonome sans avoir utilisé des sources autres que celles citées dans la bibliographie et a été vérifié par un logiciel de détection de plagiat.

L'étudiant-e accepte, le cas échéant, la clause de confidentialité.

L'utilisation des conclusions et recommandations formulées dans ce travail, sans préjuger de leur valeur, n'engage ni la responsabilité de l'étudiant-e, ni celle du/de la directeur-trice.

Fait à Genève, le 15 juin 2026

David Paulino

2 Remerciements

La partie où on remercie les gens qui nous ont aidé à réaliser ce travail de Master.

3 Résumé

Voilà le résumé de ce travail de Master. Il est important de bien résumer le travail pour que le lecteur puisse comprendre le contenu de ce travail sans avoir à le lire en entier.

Liste des tableaux

1	Comparaison des algorithmes DQN, PPO et SAC selon leurs caractéristiques fondamentales.	30
2	Hyperparamètres testés pour DQN	37
3	Hyperparamètres testés pour PPO	38
4	Hyperparamètres testés pour SAC	39
5	Score Optuna des meilleures configurations (récompense moyenne sur 20 épisodes déterministes, meilleur essai parmi 150)	41
6	Hyperparamètres optimaux — DQN / discret (score Optuna : 275.14)	42
7	Hyperparamètres optimaux — PPO / discret (score Optuna : 278.76)	43
8	Hyperparamètres optimaux — PPO / continu (score Optuna : 280.42)	44
9	Hyperparamètres optimaux — SAC / continu (score Optuna : 284.05)	45
10	Résultats de l'évaluation multi-seeds (500 000 steps, récompense moyenne sur 20 épisodes déterministes, std échantillon)	47
11	Espace d'actions dans RLGym (<i>RLGym</i> / <i>RLGym</i> , [s. d.])	51

Table des figures

1	Boucle d'interaction agent-environnement en apprentissage par renforcement	11
2	Environnement Cliff Walking.	11
3	Itérations de l'évaluation de politique dans un monde de grille 4×4 . Les états terminaux sont grisés. Inspiré de Silver (2015).	15
4	Aperçu de l'environnement LunarLander	35
5	Distribution du temps de convergence (en minutes) pour chaque combinaison algorithme/environnement, sur les 5 seeds indépendantes.	49

Table des matières

1	Déclaration	1
2	Remerciements	2
3	Résumé	3
	Liste des tableaux	4
	Table des figures	5
4	Introduction	9
5	État de l’art	10
5.1	Le paradigme Agent-Environnement	10
5.1.1	Exemple : Cliff Walking	11
5.2	Spécificités et Concepts Clés	12
5.3	Processus Décisionnel de Markov (MDP)	13
5.4	Model-Based	14
5.4.1	Policy Evaluation et Policy Iteration	14
5.4.2	Value Iteration	15
5.5	Model-Free	16
5.5.1	Monte-Carlo	17
5.5.2	Temporal-Difference (TD)	18
5.6	Apprentissage en environnement inconnu	21

5.6.1	On-Policy vs Off-Policy	21
5.6.2	Le dilemme Exploration-Exploitation	21
5.6.3	SARSA	22
5.6.4	Q-Learning	22
5.7	Du RL Tabulaire au Deep RL	23
5.7.1	Le défi de la stabilité : la Triade Mortelle	23
5.7.2	Deep Q-Network	24
5.8	Méthodes Actor-Critic	25
5.8.1	Architecture Actor-Critic	25
5.8.2	Proximal Policy Optimization (PPO)	26
5.8.3	Soft Actor-Critic (SAC)	28
5.9	Synthèse et positionnement des algorithmes étudiés	30
6	Problématique	31
7	Outils utilisés	31
7.1	Gymnasium	32
7.2	Stable Baselines3	32
7.3	Optuna	32
7.4	Weights & Biases	33
7.5	RLgym	33
7.5.1	RocketSim	33
7.5.2	RocketSimVis	33

8	Phase I - Validation méthodologique	34
8.1	Environnement et algorithmes sélectionnés	34
8.2	Optimisation des hyperparamètres	36
8.2.1	Cadre général	36
8.2.2	Élagage des essais non concluants	36
8.2.3	Hyperparamètres testés	37
8.3	Résultats de l'optimisation	41
8.3.1	DQN — LunarLander discret	41
8.3.2	PPO — LunarLander discret	42
8.3.3	PPO — LunarLanderContinuous	43
8.3.4	SAC — LunarLanderContinuous	44
8.4	Évaluation finale multi-seeds	46
8.4.1	Protocole d'évaluation finale	46
8.4.2	Résultats de l'évaluation multi-seeds	47
9	Phase II - Apprentissage sur Rocket League	50
9.1	Environnement de jeu	50
9.1.1	Espace d'actions	51
9.1.2	Espace d'observation	53
10	Discussion transversale	54
11	Conclusion	54

4 Introduction

Parmi les différents paradigmes de l'intelligence artificielle, l'apprentissage par renforcement (*Reinforcement Learning* ou RL) se distingue par sa capacité à former des agents décisionnels autonomes par le biais d'une interaction continue avec leur environnement (Sutton et al., 2015). Contrairement à l'apprentissage supervisé qui s'appuie sur des ensembles de données étiquetés, le RL repose fondamentalement sur un processus d'essais et d'erreurs. L'agent explore son environnement et affine sa *politique* d'action en se basant exclusivement sur des signaux de récompense, avec pour objectif ultime de maximiser ses gains cumulés sur le long terme.

Cette approche bio-inspirée a démontré des résultats spectaculaires dans des domaines aussi variés que la maîtrise de jeux stratégiques complexes (Zaffagni, 2022 ; *OpenAI Five*, 2018), le contrôle en robotique (Tang et al., 2024) ou encore la personnalisation des systèmes de recommandation (Iftikhar et al., 2024). Toutefois, son déploiement pose d'importants défis pratiques, notamment en ce qui concerne l'efficacité d'échantillonnage (*sample efficiency*), la formulation mathématique pertinente de la fonction de récompense, et la gestion du délicat compromis entre l'exploration de nouvelles actions et l'exploitation des connaissances déjà acquises.

5 État de l'art

Cette section introduit les fondations théoriques de l'apprentissage par renforcement, en s'appuyant principalement sur l'ouvrage de référence de Sutton et al. (2015) et les cours de Silver (2015).

5.1 Le paradigme Agent-Environnement

Le socle du RL est modélisé par une boucle d'interaction perpétuelle entre deux entités distinctes (voir Figure 1) :

- **L'agent** : le système apprenant et décisionnel.
- **L'environnement** : le système externe avec lequel l'agent interagit et qui réagit à ses actions.

À chaque pas de temps discret t , ce cycle se déroule comme suit :

1. L'agent perçoit une **observation** O_t de l'environnement et reçoit un signal scalaire de **récompense** R_t .
2. Sur la base de ces informations, l'agent sélectionne et exécute une **action** A_t dictée par sa stratégie (politique).
3. L'environnement intègre cette action, évolue vers un nouvel état, puis retourne à l'agent une nouvelle observation O_{t+1} ainsi qu'une récompense R_{t+1} évaluant la pertinence de la transition effectuée.

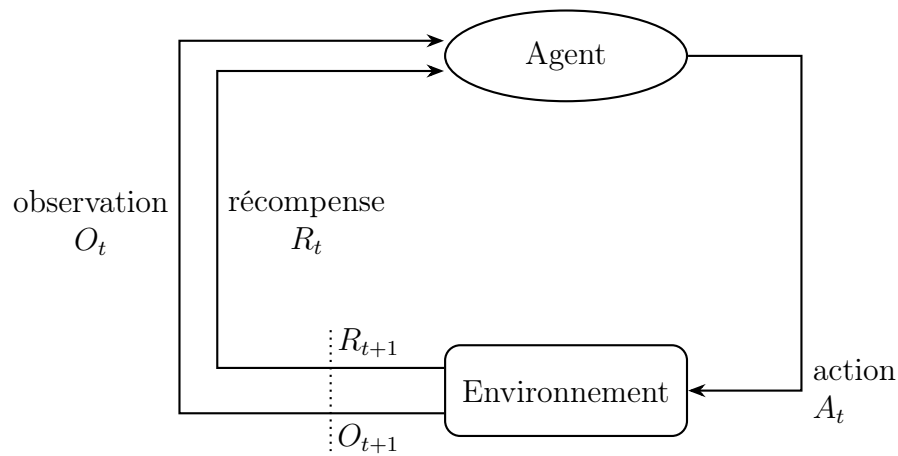


FIGURE 1 – Boucle d’interaction agent-environnement en apprentissage par renforcement

5.1.1 Exemple : Cliff Walking

L’environnement *Cliff Walking*¹ (Figure 2) illustre ces mécanismes. L’agent navigue sur une grille pour atteindre une cible en évitant une falaise. Il reçoit une récompense de -1 par pas parcouru (pour inciter à la rapidité) et de -100 avec retour au départ s’il tombe. Cet exemple met en évidence un compromis classique : l’agent doit équilibrer la minimisation de la distance pénalisante tout en évitant un risque majeur.

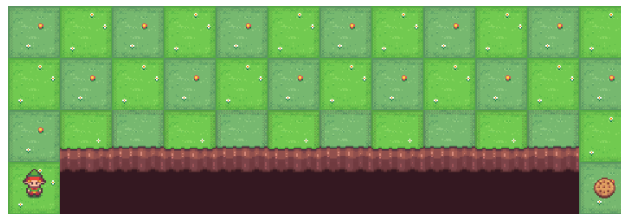


FIGURE 2 – Environnement Cliff Walking.

1. https://gymnasium.farama.org/environments/toy_text/cliff_walking/

5.2 Spécificités et Concepts Clés

Le RL se caractérise principalement par l'absence d'expert superviseur (aucun label ne vient indiquer la "bonne" action à prendre, seul le signal de récompense sert de retour), des *feedbacks* souvent différés soulignant le problème de l'assignation de crédit, et des données fortement corrélées temporellement (elles ne sont pas indépendantes et identiquement distribuées, ou i.i.d.).

Afin de formaliser ce problème, voici les cinq composantes fondamentales de la boucle d'interaction (Sutton et al., 2015) :

- **État de l'Environnement** (S_t) : L'état résume l'ensemble des informations de l'environnement pertinentes à un instant t pour que l'agent puisse déterminer son comportement. L'idéal est qu'il possède la *propriété de Markov* (c'est-à-dire qu'il englobe tout l'historique de manière compacte).
- **Action** (A_t) : La décision exécutée par l'agent dans l'état S_t . Selon le problème, l'espace des actions peut être discret (ex : aller en haut, en bas, à gauche, à droite) ou au contraire continu (ex : le degré de braquage d'un volant).
- **Récompense** (R_{t+1}) : Le signal scalaire émis par l'environnement qui évalue purement et immédiatement le succès de l'action A_t . L'objectif central du RL est d'optimiser l'accumulation temporelle de ces retours mesurables.
- **Politique** (π) : La stratégie propre à l'agent. Cette fonction détermine quelles actions entreprendre en fonction de chaque état observé. Elle peut être déterministe ($a = \pi(s)$) ou stochastique, c'est-à-dire attribuer une probabilité donnée d'exécuter une action ($P(A_t = a | S_t = s)$).
- **Fonction de valeur** (V^π, Q^π) : Mesure la qualité ("value") d'un état (ou d'une paire état-action), c'est-à-dire l'espérance mathématique des récompenses qui seront cumulées à long terme à partir de cet état, en suivant la politique π .

5.3 Processus Décisionnel de Markov (MDP)

L'environnement d'apprentissage dans sa globalité est formellement modélisé par un Processus Décisionnel de Markov (MDP). Ce puissant cadre mathématique repose sur la **propriété de Markov** : l'état actuel S_t contient toutes les informations nécessaires et suffisantes pour extrapoler le comportement futur. La prédiction de l'état suivant dépend ainsi uniquement de l'état présent, indépendamment de tout l'historique de navigation passée ($P(S_{t+1}|S_t) = P(S_{t+1}|H_t)$).

Un MDP se décrit classiquement par le système $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$ et $\mathcal{A}$ représentent respectivement l'ensemble des états que peut prendre l'environnement et l'ensemble des actions appartenant à l'agent.
- $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$ définit la matrice des probabilités de transition. Elle modélise la nature (déterministe ou probabiliste) des changements d'état résultant d'une action a spécifique.
- $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ fixe la récompense espérée en transitant par l'action a à partir de l'état s .
- $\gamma \in [0, 1]$ est le facteur d'actualisation ou de dépréciation (*discount factor*). Il quantifie l'intérêt lié à l'horizon temporel des récompenses : une valeur de γ proche de 0 réduit le temps de vue et favorise les gains très immédiats, tandis qu'une valeur se rapprochant de 1 insiste sur la conception de stratégies à long terme.

L'agent vise in fine à maximiser son **retour cumulé** global G_t , exprimé comme une série géométrique de ces récompenses actualisées :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

Pour l'exemple du labyrinthe, on pourrait imaginer une récompense $\mathcal{R}$ de -1 par étape de déplacement (pour pénaliser le temps perdu) et de +10 en trouvant la sortie, chaque case constituant un état $\mathcal{S}$, et les pas (G, D, H, B) constituant les actions $\mathcal{A}$. Le facteur γ garantit alors que l'agent privilégiera mathématiquement un chemin court et rapide vers la sortie plutôt qu'une errance prolongée.

5.4 Model-Based

Pour classer les méthodes d'apprentissage par renforcement, il est utile de distinguer les approches basées sur un modèle (*model-based*) de celles sans modèle (*model-free*). Les méthodes *model-based* supposent que l'agent possède une connaissance complète de la dynamique de l'environnement, représentée par les fonctions de transition $\mathcal{P}$ et de récompense $\mathcal{R}$.

Grâce à cette connaissance, l'agent peut planifier ses actions en évaluant mentalement leurs conséquences avant même de les exécuter. Ainsi, plutôt que d'apprendre par essais et erreurs, l'agent calcule mathématiquement la stratégie optimale dès le départ, pour autant que la complexité du problème le permette.

5.4.1 Policy Evaluation et Policy Iteration

La *Policy Iteration* (ou itération de politique) est un processus qui permet de trouver la politique optimale en alternant deux étapes clés. La première consiste à évaluer la politique actuelle pour connaître sa valeur exacte (*Policy Evaluation*). La seconde étape cherche ensuite à l'améliorer en choisissant systématiquement la meilleure action possible en fonction des résultats de l'évaluation précédente de manière *greedy* (*Policy Improvement*).

Ce processus d'évaluation suivi d'une amélioration est répété itérativement. La convergence vers la politique qualifiée d'optimale (π^*) garantit alors systématiquement $V^{\pi^*} \geq V^\pi$ quel que soit l'état initial comparé (Silver, 2015).

Mesurer la performance d'une politique donnée équivaut à quantifier l'espérance mathématique cumulative des récompenses reçues si l'agent respecte continuellement π . Pour ce faire, nous employons une récurrence itérative construite à partir d'une estimation à zéro ($V_0(s) = 0$) :

$$V_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \quad (2)$$

Au fil des itérations, les récompenses associées aux actions se propagent

d'état en état (comme l'illustre la Figure 3, où l'influence des états terminaux rayonne vers les cases adjacentes à chaque étape k). Sous réserve d'un nombre infini d'itérations, les calculs convergent et s'affinent.

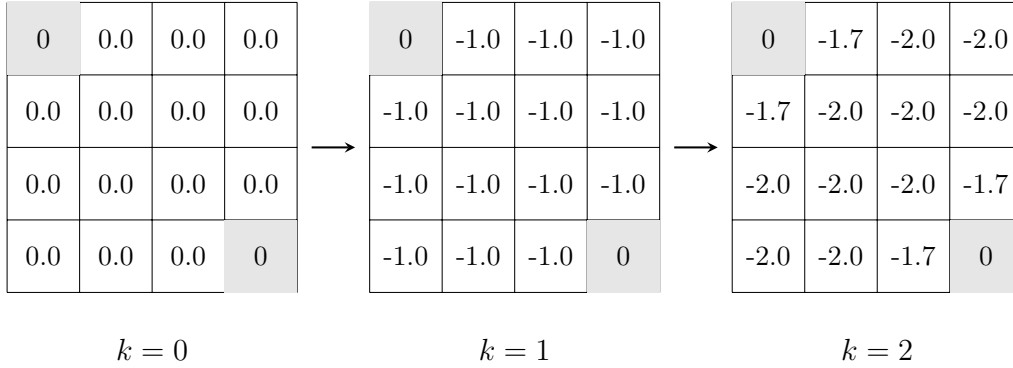


FIGURE 3 – Itérations de l'évaluation de politique dans un monde de grille 4×4 . Les états terminaux sont grisés. Inspiré de Silver (2015).

Bien qu'une évaluation parfaite soit mathématiquement faisable, l'objectif fondamental demeure l'identification rapide d'une stratégie de jeu plus favorable. Dans les faits, un nombre restreint d'itérations k d'évaluation confère une estimation suffisamment robuste pour orienter la décision opportuniste *greedy*.

La mise à jour de la politique s'oriente alors selon le critère de sélection de l'action optimale :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a) \quad (3)$$

Tant que la nouvelle politique générée diffère, ces opérations inter-cycles se relaient. Ce processus prend le nom d'**itération modifiée** lorsque l'évaluation s'interrompt pour économiser du temps de calcul dès lors qu'un seuil minime ϵ est atteint entre deux évaluations.

5.4.2 Value Iteration

Contrairement à l'itération de politique qui évalue une stratégie précise à chaque étape, la *Value Iteration* s'attaque à l'optimisation par la fusion des

deux étapes précédentes en une seule opération globale. L'algorithme cherche à trouver directement la valeur maximale de chaque état, sans avoir besoin de stocker une politique intermédiaire lors de l'entraînement.

Cette mécanique réside exclusivement dans l'introduction d'un opérateur "maximum" qui condense l'évaluation et l'amélioration :

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \quad (4)$$

Pour chaque état s et à chaque étape, l'agent met à jour ses valeurs en supposant qu'il prendra systématiquement l'action future la plus rémunératrice. En pratique, on arrête les calculs (la phase de convergence) dès que la propagation des valeurs devient marginale d'une itération à l'autre (par exemple lorsque la modification $\max_s |V_{k+1}(s) - V_k(s)| < \epsilon$).

Une fois que les valeurs de l'ensemble V sont stabilisées, l'approche pour définir de quel comportement l'agent doit faire preuve (extraire π^*) se fait intuitivement. Il suffit à l'agent de regarder les états accessibles depuis sa position actuelle et de choisir l'action lui rapportant la valeur la plus élevée :

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V^*(s') \right] \quad (5)$$

5.5 Model-Free

Cependant, l'utilisation de l'apprentissage par renforcement s'applique majoritairement à des environnements réels complexes où les règles du monde sont stochastiques, inconnues, ou trop volumineuses pour être modélisées. Sans la connaissance des formules de probabilité de transition $\mathcal{P}$ ni des fonctions de récompense $\mathcal{R}$, les méthodes *model-based* s'avèrent inapplicables car l'agent ne peut pas se projeter mathématiquement.

Pour surmonter cet obstacle, les méthodes dites *model-free* (sans modèle) effectuent un apprentissage fondé sur l'expérience. Elles s'alimentent exclusivement des données rencontrées au cours de l'interaction de l'agent avec

l'environnement pour estimer peu à peu la valeur des actions. Ce changement d'approche correspond à l'idée exprimée par Silver (2015) caractérisant ces processus comme une étape de véritable apprentissage par expérience (*learning*) plutôt que de simple planification en amont (*planning*).

5.5.1 Monte-Carlo

L'approche de Monte-Carlo (MC) s'appuie sur une idée très intuitive en ce qui concerne l'apprentissage basé sur l'expérience : la valeur d'un état correspond simplement à la moyenne de la somme des récompenses accumulées lors des récents passages par cet état. À l'inverse des méthodes *model-based* qui mettent à jour les estimations en s'appuyant sur d'autres estimations de valeurs (créant ainsi une chaîne de prédictions imbriquées), Monte-Carlo se fie uniquement aux résultats complets observés directement sans dépendre d'approximations intermédiaires.

Le fonctionnement de base des méthodes de Monte-Carlo repose sur trois prérequis. Le premier est la nécessité de générer un apprentissage par une séquence d'expériences (états, actions, et récompenses) accumulée par l'agent au cours de ses essais successifs. Par corrélation, l'environnement étudié doit obligatoirement être de nature "épisodique". C'est-à-dire qu'une interaction doit toujours avoir une fin déterminée, au contraire de processus cycliques tournant infiniment. En effet, tout le principe repose sur le fait de pouvoir calculer le retour accumulé et total de la session, ce qui requiert d'atteindre la fin de l'épisode (au *game over* d'une partie d'échecs par exemple) pour être possible. Naturellement, ces données servent une estimation qui, répétée pour chaque nouvel essai, s'affine statistiquement au fil de l'augmentation du nombre de lancers.

Pour des raisons d'efficacité, l'algorithme ne garde pas en mémoire l'intégralité des résultats passés pour calculer sa moyenne de zéro à chaque fois. Il utilise plutôt une méthode de mise à jour incrémentale. À la fin de chaque épisode, il passe en revue les états qu'il a visités et corrige directement sa dernière estimation :

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t)) \quad (6)$$

Ici, la différence ($G_t - V(S_t)$) exprime très exactement l'erreur mesurée entre le résultat du retour nouvellement calculé en finissant l'épisode (G_t) et l'estimation jusqu'alors théorisée pour l'état en passe d'être mis à jour ($V(S_t)$). Cette erreur est tempérée par un coefficient nommé α (*learning rate*), ce qui évite un bouleversement complet des connaissances en cas de coup de chance et lisse l'apprentissage.

Au cours d'un épisode, un état précis peut faire l'objet de visites répétées (comme l'agent passant par une case du labyrinthe plusieurs fois de suite). Ce détail force l'approche de l'algorithme à privilégier deux méthodes de décompte distinctes. Avec le premier traitement (*First-Visit*), l'algorithme relève seulement et ajuste la valeur basée sur ce qui s'est déroulé après avoir touché l'état la toute première fois au cours d'une occurrence. La méthode diamétralement opposée (*Every-Visit*) reconsidère l'évaluation de l'état pour chacune de ses apparitions.

Si cette méthode présente l'avantage d'être très robuste (ses estimations sont toujours basées sur des résultats réels et ne sont donc pas perturbées par de potentielles prédictions erronées sur d'autres états), elle souffre cependant de défauts majeurs dans son rythme d'apprentissage. En effet, l'agent doit systématiquement attendre la fin définitive de l'épisode pour mettre à jour ses connaissances, ce qui rend la progression particulièrement lente selon l'environnement. De plus, cela crée un problème critique de forte variance : un coup de chance exceptionnel ou un désastre va impacter le score final de tout l'épisode. Cela peut conduire à des mises à jour très versatiles et irrégulières, rendant l'apprentissage instable et difficile à maîtriser.

5.5.2 Temporal-Difference (TD)

Les méthodes des différences temporelles (TD) combinent les forces des méthodes de Monte-Carlo et de la programmation dynamique. À l'instar de Monte-Carlo, elles apprennent directement à partir de l'expérience brute, sans nécessiter de modèle de transition ($\mathcal{P}$). Cependant, à l'instar de la programmation dynamique, elles s'affranchissent de l'attente de la fin de l'épisode en utilisant le *bootstrapping*² (Sutton et al., 2015).

2. Principe consistant à mettre à jour l'estimation d'un état en se basant sur l'estimation de l'état suivant, plutôt que sur le retour final réel.

Pour illustrer cette nuance, Silver (2015) propose l'analogie du trajet en voiture. Imaginons un conducteur estimant arriver chez lui à 18h30. À 18h15, il se retrouve bloqué dans un embouteillage. Une approche Monte-Carlo obligerait le conducteur à attendre d'être effectivement garé chez lui pour constater son retard et ajuster ses prévisions futures. À l'inverse, l'approche TD permet au conducteur de "bootstrapper" : dès qu'il voit le bouchon, il actualise instantanément son estimation ("le trajet prendra finalement 50 minutes") sans avoir besoin de terminer sa route.

Mathématiquement, le mécanisme le plus simple, le TD(0), remplace le gain réel G_t par une **cible estimée** (*TD target*) composée de la récompense immédiate et de la valeur de l'état suivant :

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))}_{\text{TD target}} \quad (7)$$

La différence entre cette cible et l'estimation actuelle est nommée **erreur TD** ($\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$). Contrairement à Monte-Carlo, le TD(0) peut fonctionner dans des environnements continus ou cycliques car il apprend à chaque pas.

Par nature, le TD(0) ne regarde qu'un seul pas dans le futur avant de consulter son estimation. On peut toutefois généraliser ce concept en enregistrant n étapes de récompenses réelles avant de "bootstrapper" sur l'état S_{t+n} . On obtient alors le **retour à n pas** (*n-step return*) :

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \quad (8)$$

Il convient de noter que le terme de *bootstrapping* $\gamma^n V(S_{t+n})$ n'est valide que si l'épisode n'est pas terminé avant l'étape $t + n$. Si l'épisode se termine à $t + k$ avec $k < n$, ce terme disparaît et le retour se réduit à la somme réelle des récompenses obtenues, ce qui équivaut à l'estimateur de Monte-Carlo.

Si $n = 1$, l'approche équivaut au TD(0). Si $n \rightarrow \infty$, elle rejoint la méthode de Monte-Carlo. Le choix de n devient alors un curseur permettant de modifier l'horizon de prédiction de l'agent.

Cependant, plutôt que de choisir arbitrairement ce nombre de pas n , l'algorithme $TD(\lambda)$ propose une synthèse en unifiant simultanément tous les horizons possibles ($n = 1, 2, \dots, \infty$). Ce mélange est réalisé via une moyenne pondérée par un paramètre $\lambda \in [0, 1]$, définissant ainsi le **λ -return** (G_t^λ) :

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \quad (9)$$

Bien que le G_t^λ offre une cible robuste, il souffre du même défaut que Monte-Carlo : il regarde vers le futur et nécessite d'attendre la fin de l'épisode pour être calculé (approche dite *Forward View*). Pour rendre l'apprentissage possible en temps réel, on adopte une approche *Backward View* s'appuyant sur les **traces d'éligibilité** $E_t(s)$ (Silver, 2015).

L'équivalence mathématique entre ces deux visions permet de ne plus utiliser directement le λ -return dans la mise à jour, mais de propager l'erreur TD locale (δ_t) vers le passé. Chaque état visité laisse une "empreinte" qui s'estompe avec le temps selon le facteur λ ³. La mise à jour de la trace d'éligibilité pour chaque état s à chaque pas de temps t s'écrit ainsi :

$$E_t(s) = \gamma \lambda E_{t-1}(s) + 1(S_t = s) \quad (10)$$

Ainsi, à chaque pas de temps, la fonction de valeur de **tous les états** est ajustée proportionnellement à leur responsabilité dans l'erreur courante :

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s) \quad (11)$$

Cette mécanique est particulièrement efficace : dans un jeu vidéo, si un agent saute sur plusieurs plateformes avant de tomber dans un piège, le $TD(\lambda)$ permettra de baisser instantanément la valeur de toutes les plateformes de la séquence (beaucoup pour la dernière, un peu moins pour les précédentes) au prorata de leur responsabilité dans l'issue finale.

3. Le paramètre λ contrôle la persistance de la trace : une valeur proche de 0 restreint la mise à jour aux états les plus immédiats (approche "myope"), tandis qu'une valeur proche de 1 maintient l'empreinte des états anciens plus longtemps.

5.6 Apprentissage en environnement inconnu

Le passage de l'évaluation à l'optimisation consiste à améliorer la politique de l'agent sans connaissance préalable de la dynamique de l'environnement. Dans ce contexte *model-free*, l'utilisation de la fonction de valeur d'état $V(s)$ s'avère insuffisante : pour choisir l'action optimale, l'agent aurait besoin de connaître les probabilités de transition $\mathcal{P}$. Par conséquent, les algorithmes de contrôle se basent sur la fonction action-valeur $Q(s, a)$, permettant d'identifier directement l'action maximisant le gain attendu (Silver, 2015).

5.6.1 On-Policy vs Off-Policy

Deux paradigmes fondamentaux s'opposent dans l'apprentissage sans modèle :

- **On-Policy** : L'idée est d'apprendre "sur le tas". L'agent évalue et optimise la politique π qu'il est en train d'exécuter. Les expériences utilisées pour la mise à jour reflètent directement ses propres décisions, incluant ses tentatives d'exploration.
- **Off-Policy** : L'idée est d'apprendre en observant une autre politique. L'agent procède à l'optimisation d'une politique cible π (généralement la politique optimale) à partir d'expériences générées par une politique de comportement μ distincte. Cela permet de séparer l'exploration de l'évaluation finale (Sutton et al., 2015).

5.6.2 Le dilemme Exploration-Exploitation

Pour garantir la convergence vers une solution idéale, l'agent ne peut se contenter d'être *greedy* dès le départ, au risque de rester piégé dans un optimum local. Il doit arbitrer entre l'exploitation de ses connaissances actuelles (issues de l'évaluation) et l'exploration de nouvelles stratégies. La méthode la plus répandue est la stratégie ϵ -greedy :

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{m} & \text{si } a = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a) \\ \frac{\epsilon}{m} & \text{sinon} \end{cases} \quad (12)$$

En pratique, on utilise un *scheduler* pour réduire progressivement la valeur de ϵ au fil du temps. Cette décroissance permet de satisfaire les conditions **GLIE**⁴ : l'agent explore massivement au début, puis stabilise son comportement vers une politique déterministe optimale une fois l'espace état-action suffisamment parcouru. Cependant, un *epsilon* qui descendrait trop vite à 0 créerait un manque d'exploration, empêchant l'agent de découvrir l'intégralité des récompenses et biaisant l'optimisation.

5.6.3 SARSA

L'algorithme SARSA est une méthode d'optimisation **On-Policy**. Il met à jour la fonction Q en utilisant la récompense immédiate et l'évaluation de l'action suivante (A') effectivement choisie par la politique actuelle de l'agent. Son nom illustre d'ailleurs la séquence (S, A, R, S', A') utilisée pour la mise à jour :

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A)) \quad (13)$$

Parce qu'il inclut l'action réelle A' de l'agent (et donc les erreurs potentielles liées à l'exploration) dans son calcul, SARSA réalise une optimisation "prudente". Il évalue les zones de l'environnement en tenant compte du fait qu'un mouvement aléatoire dû à ϵ pourrait entraîner une chute brutale de la récompense (Silver, 2015).

5.6.4 Q-Learning

À l'inverse, le Q-Learning est une méthode d'optimisation **Off-Policy**. Il s'affranchit de l'action réellement choisie par l'agent pour la mise à jour en utilisant l'opérateur *maximum*. Il optimise directement la valeur de la politique optimale, indépendamment du comportement exploratoire :

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_{a'} Q(S', a') - Q(S, A)) \quad (14)$$

Ici, par rapport à SARSA, la mise à jour se base sur l'évaluation de la meilleure action possible (max), sans tenir compte de l'action A_{t+1} réellement effectuée par l'agent. Cette approche permet ainsi de converger vers la

4. *Greedy in the Limit with Infinite Exploration*

politique optimale via l'équation d'optimalité de Bellman, même si l'agent continue d'explorer activement son environnement.

5.7 Du RL Tabulaire au Deep RL

Les méthodes tabulaires présentées précédemment, bien que robustes, se heurtent à la *malédiction de la dimensionnalité*. Dans des environnements complexes (qu'il s'agisse de flux de pixels ou de vecteurs d'états continus), le nombre de paires (s, a) devient trop vaste pour être stocké ou exploré exhaustivement.

Pour surmonter cette limite, l'**approximation de fonction** est une idée centrale. L'objectif n'est plus de maintenir un tableau de valeurs exactes, mais d'apprendre une fonction paramétrée $\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$ ⁵, capable de généraliser les connaissances acquises vers des états jamais rencontrés.

5.7.1 Le défi de la stabilité : la Triade Mortelle

Le *Deep Reinforcement Learning* (DRL) exploite la puissance des réseaux de neurones profonds comme approximateurs de fonction universels. Cependant, l'intégration de ces réseaux dans une boucle d'optimisation *model-free* est intrinsèquement instable. Cette instabilité est théorisée par la **Triade Mortelle** (*Deadly Triad*) (Sutton et al., 2015), qui survient lorsque trois éléments sont réunis :

- **L'approximation de fonction** (utilisation de réseaux de neurones) ;
- **Le bootstrapping** (mise à jour d'estimations basées sur d'autres estimations) ;
- **L'apprentissage Off-Policy** (utilisation de données issues de politiques antérieures).

Sous ces conditions, la convergence des algorithmes est menacée par une forte corrélation temporelle des données et une cible de calcul mouvante (le réseau tente de rattraper une valeur qu'il modifie lui-même).

5. $\mathbf{w}$ étant un réseau de neurones

5.7.2 Deep Q-Network

L'algorithme **Deep Q-Network (DQN)** (Mnih et al., 2015) a marqué l'avènement du DRL en proposant deux solutions techniques majeures pour briser cette triade et stabiliser l'optimisation :

1. **Experience Replay** : Au lieu d'apprendre de manière séquentielle sur la donnée immédiate, l'agent stocke ses transitions (s, a, r, s') dans un tampon mémoire (*Replay Buffer*). Il échantillonne ensuite aléatoirement des "mini-batches" d'expériences passées pour s'entraîner. Cela casse la corrélation temporelle des trajectoires et rend l'apprentissage plus stable en se rapprochant des conditions *i.i.d.* (indépendantes et identiquement distribuées) de l'apprentissage supervisé.
2. **Target Network** : Pour éviter que la cible de mise à jour ne change à chaque itération, DQN utilise deux réseaux distincts. Un réseau "online" (paramétré par $\mathbf{w}$) qui est optimisé activement, et un réseau "cible" (*target*, paramétré par $\mathbf{w}^-$) dont les poids sont gelés et périodiquement synchronisés avec le premier.

L'apprentissage s'effectue en minimisant l'erreur quadratique moyenne entre la prédiction du réseau online et la cible fournie par le réseau target via une étape de descente de gradient. La fonction de perte (*loss*) s'écrit formellement :

$$L(\mathbf{w}) = \mathbb{E}_{(s,a,r,s') \sim \text{Replay}} \left[\left(r + \gamma \max_{a'} \hat{q}(s', a', \mathbf{w}^-) - \hat{q}(s, a, \mathbf{w}) \right)^2 \right] \quad (15)$$

Cette architecture permet de converger efficacement dans des environnements à large espace d'états (comme le traitement d'images/pixels de jeux vidéo). Il est toutefois fondamental de retenir que l'algorithme DQN est intrinsèquement restreint aux **espaces d'actions discrets**. En effet, le calcul de la cible repose sur l'évaluation de la meilleure action future via l'opérateur $\max_{a'} \hat{q}(s', a')$. Si l'espace des actions était continu, rechercher algorithmiquement le maximum global d'une fonction neuronale hautement non linéaire à chaque itération et pour chaque transition représenterait un coût informatique colossal et prohibitif.

Pour pallier cette limitation, DDPG (*Deep Deterministic Policy Gradient*) (Lillicrap et al., 2015) étend les principes de DQN aux espaces continus en

apprenant simultanément une politique déterministe (l'*Acteur*) et une fonction de valeur (le *Critique*), supprimant ainsi la nécessité de l'opérateur max. DDPG constitue la fondation architecturale directe de SAC, qui en corrige les principaux problèmes de stabilité et d'exploration, comme détaillé dans la section suivante.

5.8 Méthodes Actor-Critic

Pour pallier l'incompatibilité de l'algorithme DQN avec les espaces d'actions continus (comme les couples moteurs de la robotique ou le pilotage de véhicules), une approche radicalement différente est exploitée : le **Gradient de Politique** (*Policy Gradient*).

L'idée est de s'affranchir de la recherche de la valeur maximale des actions pour dicter le comportement. On y optimise plutôt directement les paramètres θ d'une politique stochastique $\pi_\theta(a|s)$ par montée de gradient, dans l'optique de maximiser le retour algorithmique attendu $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$.

5.8.1 Architecture Actor-Critic

Les méthodes modernes de l'état de l'art fusionnent l'apprentissage de la fonction de valeur et l'optimisation directe de la politique dans une architecture hybride nommée **Actor-Critic** (Sutton et al., 2015) :

- **L'Acteur** ($\pi_\theta(a|s)$) : Un réseau de neurones constituant la politique. Il décide quelle action entreprendre.
- **Le Critique** ($V_w(s)$ ou $Q_w(s, a)$) : Un second réseau agissant en tant qu'évaluateur. Il calcule l'**avantage** estimé d'une action, défini comme $\hat{A}(s, a) = Q(s, a) - V(s)$, qui traduit si l'action choisie s'est avérée meilleure ou pire que ce qui était attendu en moyenne pour cet état. Soustraire $V(s)$ joue ici le rôle de **baseline** : ce résultat fondamental, établi par Williams (1992), démontre qu'une baseline ne biaise pas l'estimation du gradient de politique tout en réduisant significativement sa variance. Sans cette soustraction, le signal de mise à jour de l'Acteur serait extrêmement bruité, rendant l'apprentissage instable. Le Critique offre ainsi un signal qualitatif clair et à faible variance pour guider la mise à jour de l'Acteur.

L'avantage majeur de ce duo est qu'il paramètre intrinsèquement très bien la nature des actions (discrètes ou continues). Pour un problème *discret*, la couche de sortie de l'Acteur applique une fonction *Softmax* définissant la probabilité de tirer chaque action de la liste. Pour un espace *continu*, l'Acteur va générer non pas des actions brutes, mais les paramètres statistiques d'une loi de distribution continue (typiquement la moyenne μ et l'écart-type σ d'une variable gaussienne), au sein de laquelle l'action réelle sera piochée aléatoirement.

5.8.2 Proximal Policy Optimization (PPO)

L'algorithme **PPO** (Schulman ; Wolski et al., 2017) est devenu un standard industriel du RL pour son équilibre entre simplicité d'implémentation et grande robustesse empirique.

Estimation de l'avantage via GAE. PPO est quasi-systématiquement combiné avec l'estimateur **GAE** (*Generalized Advantage Estimation*) (Schulman ; Moritz et al., 2018), qui généralise le retour à n pas pour l'estimation de l'avantage. Plutôt que de fixer un horizon n arbitraire, GAE calcule une moyenne pondérée exponentiellement de toutes les erreurs TD successives, contrôlée par un paramètre $\lambda_{\text{GAE}} \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda_{\text{GAE}})} = \sum_{l=0}^{\infty} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l} \quad (16)$$

où $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ est l'erreur TD locale. λ_{GAE} joue le même rôle que λ dans $\text{TD}(\lambda)$: une valeur proche de 0 produit une estimation à faible variance mais potentiellement biaisée (horizon court), tandis qu'une valeur proche de 1 réduit le biais au prix d'une variance plus élevée (horizon long). Ce paramètre sera directement retrouvé lors de la configuration expérimentale de PPO (section 8.2.3).

Clipped Surrogate Objective. Un problème fondamental du gradient de politique est que des mises à jour trop importantes peuvent détruire une

politique fonctionnelle de manière irréversible. PPO sécurise l'apprentissage en introduisant une fonction objectif **écrêtée** (*Clipped Surrogate Objective*) qui limite strictement l'amplitude des modifications entre deux mises à jour de la politique :

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (17)$$

Les trois termes de cette expression (que l'on cherche à maximiser) s'interprètent comme suit :

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ est le ratio de probabilité. Il quantifie si l'action a_t est plus ($r_t > 1$) ou moins ($r_t < 1$) probable sous la nouvelle politique que sous l'ancienne.
- $\hat{A}_t$ est l'avantage estimé par le Critique, indiquant si l'action s'est révélée meilleure ou pire que l'espérance de l'état.
- La fonction clip borne ce ratio dans l'intervalle $[1 - \varepsilon, 1 + \varepsilon]$ (typiquement $\varepsilon = 0.2$), supprimant toute incitation à s'en écarter davantage.

L'opérateur min garantit une politique conservatrice dans les deux sens : si une action a été bonne ($\hat{A}_t > 0$), on cherche à augmenter sa probabilité, mais uniquement dans la limite autorisée ; si elle a été mauvaise ($\hat{A}_t < 0$), on la pénalise, là encore dans la même fenêtre bornée.

Notion de rollout et efficacité des données. En pratique, PPO collecte un **rollout** — une séquence de **n_steps** transitions consécutives — avant d'effectuer ses mises à jour. Ce batch de données est exploité sur plusieurs passes d'optimisation (**n_epochs**), découpé en mini-batches de taille **batch_size**. Pour un rollout de 2048 steps, des mini-batches de 256 et 20 époques, PPO effectue ainsi $\lfloor 2048/256 \rfloor \times 20 = 160$ mises à jour du réseau par rollout. Ce mécanisme compense partiellement la *sample efficiency* structurellement limitée des méthodes *on-policy* ; néanmoins, le batch doit être intégralement renouvelé après chaque cycle de mise à jour, car les données générées par la politique modifiée ne seraient plus représentatives de la politique ayant produit les estimations d'avantage (Schulman ; Wolski et al., 2017).

5.8.3 Soft Actor-Critic (SAC)

SAC (Haarnoja; Zhou; Abbeel et al., 2018) étend le cadre actor-critic en intégrant l'**entropie** de la politique à la fonction objectif. L'agent cherche à maximiser simultanément les récompenses accumulées et le caractère aléatoire de sa politique, afin de rester exploratoire et d'éviter une convergence prématurée vers une stratégie sous-optimale :

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right] \quad (18)$$

où $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a|s_t)]$ est l'entropie de Shannon de la politique en l'état s_t , et $\alpha > 0$ est un coefficient de température pondérant l'importance relative de l'entropie.

Ajustement automatique du coefficient d'entropie. Le coefficient α peut être fixé manuellement ou ajusté automatiquement. Haarnoja; Zhou; Hartikainen et al. (2019) proposent d'optimiser α à chaque step de gradient pour maintenir une entropie cible $\mathcal{H}^*$, définie par défaut comme l'opposé de la dimension de l'espace d'action :

$$\mathcal{H}^* = -\dim(\mathcal{A}) \quad (19)$$

Ce choix correspond approximativement à l'entropie d'une distribution uniforme sur un espace d'action normalisé, garantissant une exploration suffisante sans sur-contraindre la politique.

Reparameterization trick. SAC exploite le **reparameterization trick** pour rendre son optimisation différentiable. Échantillonner directement une action aléatoire $a \sim \pi_\theta(\cdot|s)$ est une opération non différentiable qui bloque la rétropropagation du gradient.

State Dependent Exploration (SDE). Par défaut, SAC rééchantillonne un bruit gaussien indépendant à chaque pas de temps, ce qui peut produire une exploration sans cohérence temporelle. Raffin et al. (2021) proposent une alternative, l'**State Dependent Exploration (SDE)** : plutôt que de tirer un nouveau bruit à chaque step, SDE maintient un vecteur de bruit ε fixe pendant plusieurs steps consécutifs et apprend un mapping $f(s, \varepsilon)$ dépendant de l'état courant. L'exploration devient ainsi cohérente dans le temps — l'agent explore des trajectoires entières plutôt que des actions isolées — et structurée par l'état, ce qui est particulièrement adapté aux tâches de contrôle continu nécessitant des séquences d'actions coordonnées.

Double réseau critique. Pour contenir le biais de surestimation des valeurs Q (*overestimation bias*) — problème structurel des méthodes actor-critic identifié par Fujimoto et al. (2018) — SAC adopte le principe du **double réseau critique** hérité de TD3 : deux réseaux Q_{ϕ_1} et Q_{ϕ_2} sont entraînés indépendamment sur les mêmes transitions, et la cible de mise à jour utilise systématiquement le minimum des deux estimations :

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}'|s'), \quad \tilde{a}' \sim \pi_{\theta}(\cdot|s') \quad (20)$$

En prenant le minimum, on pénalise les états sur-estimés plutôt que de les amplifier, ce qui stabilise l'apprentissage du critique et, par ricochet, celui de l'acteur.

Soft target update. Contrairement à DQN qui synchronise périodiquement son réseau cible par copie intégrale des poids, SAC adopte une mise à jour **douce** (*soft update*) à chaque step de gradient pour chacun de ses deux réseaux cibles :

$$\phi_{\text{cible}} \leftarrow \tau \phi + (1 - \tau) \phi_{\text{cible}} \quad (21)$$

où $\tau \ll 1$ (valeur standard : 0.005) est le taux d'interpolation. Cette approche produit une cible évoluant lentement et continûment, offrant un signal d'apprentissage plus stable que les mises à jour discrètes de DQN. C'est

l'ensemble de cette architecture — deux critiques, deux réseaux cibles, un acteur, et le mécanisme d'entropie adaptative — qui explique généralement le coût computationnel structurellement plus élevé de SAC par rapport à des algorithmes n'entretenant qu'un seul réseau critique.

5.9 Synthèse et positionnement des algorithmes étudiés

Les trois algorithmes retenus pour cette étude ont été délibérément choisis pour couvrir des paradigmes d'apprentissage complémentaires. Le tableau 1 résume leurs caractéristiques fondamentales selon les axes de comparaison qui structureront l'analyse expérimentale.

TABLE 1 – Comparaison des algorithmes DQN, PPO et SAC selon leurs caractéristiques fondamentales.

Critère	DQN	PPO	SAC
Paradigme	Off-Policy	On-Policy	Off-Policy
Espace d'actions	Discret	Discret / Continu	Continu
Replay Buffer	✓	×	✓
<i>Sample Efficiency</i>	Moyenne	Faible	Élevée
Stabilité	Moyenne	Élevée	Élevée
Exploration	ϵ -greedy	Stochastique (distribution apprise)	Maximisation d'entropie
Référence fondatrice	Mnih et al. (2015)	Schulman ; Wolski et al. (2017)	Haarnoja ; Zhou ; Abbeel et al. (2018)

Note — L'exploration de PPO est intrinsèquement stochastique : la politique paramétrise une distribution de probabilités (catégorielle en discret, gaussienne en continu) depuis laquelle les actions sont échantillonnées à chaque step. Contrairement à DQN, aucun mécanisme ϵ -greedy n'est nécessaire. Le paramètre ϵ de PPO désigne le seuil d'écrêtage du Clipped Surrogate Objective, sans lien avec la stratégie d'exploration.

Cette diversité de paradigmes permet une comparaison expérimentale riche : DQN sert de référence tabulaire profonde, PPO représente l'approche On-Policy de l'état de l'art, et SAC incarne l'efficacité Off-Policy avec exploration intrinsèque. Leur confrontation sur des environnements Gymnasium standardisés, puis sur un environnement complexe avec *reward shaping*, constituera le cœur de l'analyse empirique de ce mémoire.

6 Problématique

L'apprentissage par renforcement (RL) est un domaine riche mais dont la mise en pratique soulève de nombreux défis techniques. Ce mémoire a pour objectif d'explorer concrètement ce domaine à travers deux grands axes.

Le premier axe concerne le problème d'optimisation et de sélection des modèles. Face à un problème donné, plusieurs décisions cruciales s'imposent : comment choisir d'abord le bon algorithme (DQN, PPO, SAC, etc.) en fonction des spécificités de l'environnement (comme des espaces d'actions discrets ou continus) ? Ensuite, comment relever le défi de l'optimisation des hyperparamètres, étape indispensable pour faire converger ces modèles au sein d'environnement ?

Le second axe se concentre sur l'alignement du comportement de l'agent avec les attentes du concepteur à travers le *reward shaping* (l'ingénierie de la récompense). Dans un environnement complexe, définir la récompense s'avère extrêmement délicat. L'enjeu est ici de comprendre les problématiques liées à cette conception : comment construire un signal qui guide efficacement l'apprentissage, tout en évitant les comportements opportunistes inattendus (*reward hacking*) liés à une fonction mal spécifiée.

Ainsi, la problématique centrale de ce travail est de comprendre la méthodologie pratique du RL : de la sélection et du réglage des algorithmes sur des cas d'étude standard, jusqu'aux défis complexes de la conception des récompenses (*reward shaping*) nécessaires pour résoudre des environnements avancés.

7 Outils utilisés

Dans ce mémoire, plusieurs outils et bibliothèques ont été mobilisés pour conduire les expériences d'apprentissage par renforcement.

7.1 Gymnasium

Gymnasium (anciennement OpenAI Gym) est une bibliothèque de référence pour la création et l'utilisation d'environnements d'apprentissage par renforcement. Elle expose une interface standardisée permettant d'interagir de manière uniforme avec des environnements très variés, ce qui facilite le développement et la comparaison d'algorithmes de RL. Gymnasium propose un large catalogue d'environnements, allant des jeux classiques aux tâches de contrôle continu, ce qui en fait un socle incontournable pour tester et évaluer les performances des agents. (*Gymnasium Documentation*, 2025)

7.2 Stable Baselines3

Stable Baselines3 est une bibliothèque de référence pour l'implémentation d'algorithmes d'apprentissage par renforcement reposant sur PyTorch. Elle fournit des implémentations robustes et éprouvées d'algorithmes populaires tels que DQN, PPO ou SAC. Conçue pour allier simplicité d'utilisation et hautes performances, elle constitue un choix naturel aussi bien pour la recherche que pour des applications pratiques en RL. (RAFFIN et al., 2026)

7.3 Optuna

Optuna est une bibliothèque d'optimisation d'hyperparamètres permettant d'identifier efficacement les configurations les plus performantes pour un modèle donné. Contrairement aux approches exhaustives de type *grid search*, qui parcourent l'espace des hyperparamètres de manière séquentielle et aveugle, Optuna s'appuie sur une recherche bayésienne pour orienter intelligemment l'exploration vers les régions les plus prometteuses.

Par ailleurs, Optuna intègre un mécanisme d'élagage (*pruning*) qui permet d'interrompre prématurément les essais peu prometteurs, sur la base d'une évaluation continue des performances en cours d'entraînement. Cette double approche — échantillonnage guidé et élagage précoce — s'avère particulièrement précieuse en apprentissage par renforcement, où l'entraînement des agents est à la fois coûteux en ressources de calcul et très sensible aux choix

d’hyperparamètres. (Akiba et al., 2019)

7.4 Weights & Biases

Weights & Biases (WandB) est une plateforme de suivi et de visualisation des expériences d’apprentissage automatique. Elle permet d’enregistrer les métriques d’entraînement, de visualiser les courbes d’apprentissage et de comparer les performances de différents modèles et configurations. WandB facilite par ailleurs la collaboration en centralisant l’ensemble des résultats expérimentaux et en offrant des outils de partage adaptés au travail en équipe. (*Weights & Biases*, [s. d.])

7.5 RLgym

RLgym est une bibliothèque de simulation d’environnements pour l’apprentissage par renforcement, initialement conçue pour le jeu Rocket League, mais désormais indépendante de celui-ci. Elle s’appuie sur les abstractions de Gymnasium et propose des outils spécifiques à la simulation de physique de jeu pour l’entraînement d’agents. (*RLGym / RLGym*, [s. d.])

7.5.1 RocketSim

RocketSim est une bibliothèque de simulation physique de Rocket League, optimisée pour les scénarios d’entraînement intensif. Son principal atout réside dans la prise en charge du *headless training* — un entraînement sans interface graphique — qui supprime toute charge de rendu visuel et accélère considérablement les cycles d’expérimentation. (ZealanL, 2022)

7.5.2 RocketSimVis

RocketSimVis est une bibliothèque de visualisation complémentaire à RocketSim. Elle permet de rejouer et d’inspecter visuellement les parties simulées,

ce qui s'avère précieux pour analyser les comportements des agents et appréhender les dynamiques de l'environnement. (ZealanL, 2024)

8 Phase I - Validation méthodologique

Avant d'aborder des environnements plus complexes, cette phase préliminaire vise à maîtriser les outils et pratiques du RL moderne : optimisation systématique des hyperparamètres, monitoring de la convergence et élagage automatique des configurations non prometteuses. L'environnement *LunarLander* de la suite OpenAI Gymnasium constitue un terrain d'entraînement standard et bien documenté pour valider ces pratiques.

8.1 Environnement et algorithmes sélectionnés

LunarLander (*Gymnasium Documentation - Lunar Lander*, [s. d.]) est un problème de contrôle classique dans lequel un agent doit poser un module lunaire entre deux drapeaux situés en (0,0) en gérant sa poussée et son orientation. Deux variantes sont disponibles :

- **LunarLander (discret)** : l'agent choisit à chaque pas de temps parmi quatre actions discrètes (ne rien faire, allumer le propulseur gauche, principal ou droit).
- **LunarLanderContinuous (continu)** : l'espace d'action est continu et bidimensionnel, correspondant à l'intensité de chaque propulseur.

L'espace d'observation est identique dans les deux cas : un vecteur de huit variables continues décrivant la position, la vitesse, l'angle, la vitesse angulaire et le contact des deux pattes avec le sol.

La fonction de récompense pénalise la consommation de carburant et la distance à la zone d'atterrissage, et accorde un bonus lors du contact des pattes avec le sol ainsi qu'une récompense de +100 pour un atterrissage réussi (ou une pénalité de -100 pour un crash). L'épisode se termine soit à l'atterrissage, soit au crash, soit lors d'une sortie du terrain de jeu. Un épisode est considéré **résolu** lorsque la récompense atteint ≥ 200 .

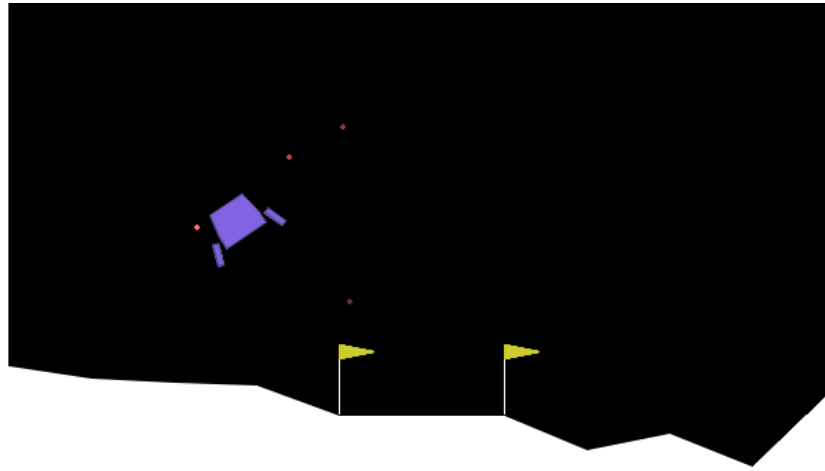


FIGURE 4 – Aperçu de l’environnement LunarLander

Ce choix se justifie par plusieurs raisons : LunarLander est suffisamment complexe pour nécessiter un réglage soigneux des hyperparamètres, disponible en variantes discrète et continue (permettant de comparer des algorithmes de natures différentes), et constitue un benchmark de référence largement utilisé dans la littérature, facilitant la mise en perspective des résultats.

Trois algorithmes ont été sélectionnés pour couvrir les principales familles du RL profond :

- **DQN** (*Deep Q-Network*) (Mnih et al., 2015) : algorithme *off-policy* basé sur la Q -valeur, applicable uniquement aux espaces d’action discrets. Évalué sur la variante *LunarLander*.
- **PPO** (*Proximal Policy Optimization*) (Schulman ; Wolski et al., 2017) : algorithme *on-policy* de type acteur-critique, applicable aux deux types d’espaces d’action. Évalué sur les deux variantes.
- **SAC** (*Soft Actor-Critic*) (Haarnoja ; Zhou ; Abbeel et al., 2018) : algorithme *off-policy* à entropie maximale, conçu pour les espaces d’action continus. Évalué sur la variante *LunarLanderContinuous*.

Ce choix permet de comparer un algorithme à valeur (DQN), un algorithme *on-policy* (PPO) et un algorithme *off-policy* à entropie maximale (SAC), qui représentent trois approches fondamentalement différentes du problème d’apprentissage par renforcement.

8.2 Optimisation des hyperparamètres

8.2.1 Cadre général

L'optimisation des hyperparamètres est réalisée avec le framework **Optuna** (Akiba et al., 2019), en utilisant le sampler *Tree-structured Parzen Estimator* (TPE). TPE est un algorithme bayésien qui modélise séparément la distribution des bons et des mauvais hyperparamètres, et dirige la recherche vers les régions prometteuses de l'espace de configuration. Chaque configuration est évaluée sur **250 000 pas de temps** d'entraînement, suivie d'une évaluation déterministe sur **20 épisodes**. La métrique optimisée est la récompense moyenne obtenue lors de cette évaluation finale.

8.2.2 Élagage des essais non concluants

Afin de réduire le coût computationnel, un **MedianPruner** est appliqué : il interrompt prématurément tout essai dont la performance intermédiaire (évaluée toutes les 10 000 steps) est inférieure à la médiane des essais précédents au même stade d'entraînement. Deux paramètres encadrent ce mécanisme :

- **n_startup_trials = 15** : les 15 premiers essais sont toujours complétés afin de fournir suffisamment de données au pruner pour établir une médiane de référence fiable avant d'activer l'élagage.
- **n_warmup_steps = 30 000** : au sein d'un essai, l'élagage ne peut s'activer qu'après 30 000 steps, laissant le temps à l'agent d'amorcer sa convergence avant d'être jugé.

Un essai interrompu par le pruner est enregistré avec l'état **PRUNED** dans la base Optuna et comptabilisé dans le total des essais effectués, au même titre qu'un essai complet. Seuls les essais ayant échoué pour des raisons techniques (crash, erreur mémoire) ne sont pas comptabilisés. La campagne cible au minimum **150 essais** par combinaison algorithme/environnement.

8.2.3 Hyperparamètres testés

Les tableaux suivants décrivent les hyperparamètres testés pour chaque algorithme, leur rôle et les plages de recherche utilisées. Pour l'ensemble des expériences, la politique neuronale repose sur des perceptrons multicouches (MlpPolicy dans Stable-Baselines3). L'espace d'observation de LunarLander étant un vecteur de huit dimensions, l'usage de réseaux convolutifs n'est pas pertinent dans ce contexte.

TABLE 2 – Hyperparamètres testés pour DQN

Hyperparamètre	Rôle	Plage / valeurs
<code>learning_rate</code>	Pas de gradient de l'optimiseur Adam (<i>continu, log</i>). Une valeur trop élevée déstabilise l'entraînement ; trop faible, la convergence est lente.	$[10^{-5}, 10^{-3}]$
<code>buffer_size</code>	Taille du <i>replay buffer</i> (<i>catégoriel</i>). Un buffer plus grand améliore la diversité des transitions échantillonnées, au coût d'une empreinte mémoire plus importante.	$\{10^4, 5 \cdot 10^4, 10^5\}$
<code>learning_starts</code>	Nombre de pas avant le début des mises à jour (<i>catégoriel</i>). Permet de pré-remplir le buffer avec des transitions aléatoires pour stabiliser les premières itérations.	$\{0, 1000, 5000\}$
<code>batch_size</code>	Nombre de transitions échantillonnées à chaque mise à jour (<i>catégoriel</i>).	$\{32, 64, 128, 256\}$
<code>gamma</code>	Facteur d'actualisation des récompenses futures (<i>catégoriel</i>). Proche de 1 : l'agent est prévoyant ; proche de 0 : il privilégie les récompenses immédiates.	$\{0.90, \dots, 0.999\}$
<code>train_freq</code>	Fréquence des mises à jour en nombre de pas de temps (<i>catégoriel</i>).	$\{1, 4, 8, 16\}$

Suite page suivante

Suite du tableau 2

Hyperparamètre	Rôle	Plage / valeurs
<code>target_update_interval</code>	Nombre de mises à jour entre deux synchronisations du réseau cible (<i>catégoriel</i>). Le <i>target network</i> stabilise l'apprentissage en fixant temporairement les valeurs cibles.	{100, 250, 500, 1000}
<code>exploration_fraction</code>	Fraction du training durant laquelle ε décroît de sa valeur initiale à sa valeur finale (<i>continu</i>).	[0.05, 0.5]
<code>exploration_final_eps</code>	Valeur minimale de ε après la phase d'exploration (<i>continu</i>). Correspond au taux d'actions aléatoires résiduel en exploitation.	[0.01, 0.1]
<code>net_arch</code>	Architecture du réseau MLP — nombre et taille des couches cachées (<i>catégoriel</i>).	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

TABLE 3 – Hyperparamètres testés pour PPO

Hyperparamètre	Rôle	Plage / valeurs
<code>learning_rate</code>	Pas de gradient de l'optimiseur Adam (<i>continu, log</i>).	$[10^{-5}, 10^{-3}]$
<code>n_steps</code>	Nombre de pas collectés avant chaque mise à jour de la politique, <i>rollout length</i> (<i>catégoriel</i>). Doit être supérieur à <code>batch_size</code> — les configurations invalides sont rejetées automatiquement lors du sampling.	{256, 512, 1024, 2048}
<code>batch_size</code>	Taille des mini-batches utilisés pour les mises à jour (<i>catégoriel</i>).	{32, 64, 128, 256, 512}
<code>n_epochs</code>	Nombre de passes sur les données collectées à chaque mise à jour (<i>catégoriel</i>). Augmenter ce nombre améliore l'utilisation des données mais peut déstabiliser la politique.	{1, 3, 5, 10, 20}

Suite page suivante

Suite du tableau 3

Hyperparamètre	Rôle	Plage / valeurs
gamma	Facteur d'actualisation des récompenses futures (<i>catégoriel</i>).	$\{0.90, \dots, 0.999\}$
gae_lambda	Paramètre λ du <i>Generalized Advantage Estimation</i> (GAE) (<i>catégoriel</i>). Contrôle le compromis biais-variance dans l'estimation de l'avantage.	$\{0.80, \dots, 1.0\}$
ent_coef	Coefficient de la prime d'entropie (<i>continu, log</i>). Encourage l'exploration en pénalisant les politiques trop déterministes.	$[10^{-8}, 0.1]$
clip_range	Paramètre ε de la contrainte PPO (<i>catégoriel</i>). Limite l'écart entre l'ancienne et la nouvelle politique lors des mises à jour.	$\{0.1, 0.2, 0.3, 0.4\}$
max_grad_norm	Norme maximale du gradient — <i>gradient clipping</i> (<i>continu, log</i>). Préviend les mises à jour explosives et stabilise l'entraînement.	$[0.3, 5.0]$
vf_coef	Coefficient de la perte du critique (<i>continu</i>). Contrôle l'importance relative de l'erreur de prédiction de valeur par rapport à la perte de politique.	$[0.1, 1.0]$
net_arch	Architecture du réseau MLP acteur-critique (<i>catégoriel</i>).	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

TABLE 4 – Hyperparamètres testés pour SAC

Hyperparamètre	Rôle	Plage / valeurs
learning_rate	Pas de gradient commun aux réseaux acteur et critique (<i>continu, log</i>).	$[10^{-5}, 10^{-3}]$

Suite page suivante

Suite du tableau 4

Hyperparamètre	Rôle	Plage / valeurs
<code>buffer_size</code>	Taille du <i>replay buffer</i> (<i>catégoriel</i>).	$\{10^4, 5 \cdot 10^4, 10^5\}$
<code>learning_starts</code>	Nombre de pas avant le début des mises à jour (<i>catégoriel</i>).	$\{1000, 5000, 10000\}$
<code>batch_size</code>	Taille des mini-batches pour les mises à jour (<i>catégoriel</i>).	$\{32, 64, 128, 256, 512\}$
<code>gamma</code>	Facteur d'actualisation des récompenses futures (<i>catégoriel</i>).	$\{0.90, \dots, 0.999\}$
<code>tau</code>	Taux de mise à jour <i>soft</i> du réseau cible (<i>catégoriel</i>). À chaque pas : $\theta_{\text{cible}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{cible}}$.	$\{0.001, 0.005, 0.01, 0.02, 0.05\}$
<code>ent_coef</code>	Coefficient d'entropie géré automatiquement (auto) : SAC ajuste ce coefficient à chaque pas de gradient pour maintenir une entropie cible $\mathcal{H}^* = -\dim(\mathcal{A})$, définie par Haarnoja ; Zhou ; Hartikainen et al., 2019 comme l'opposé de la dimension de l'espace d'action.	auto
<code>use_sde</code>	Active l' <i>State Dependent Exploration</i> (SDE) (<i>catégoriel</i>). Le bruit d'exploration dépend de l'état courant ce qui peut améliorer l'exploration dans les espaces d'action continus. Ce bruit est maintenu pendant plusieurs steps consécutifs. Des trajectoires entières peuvent ainsi être explorées.	$\{\text{True}, \text{False}\}$
<code>target_update_interval</code>	Nombre de gradient steps entre deux mises à jour du réseau cible (<i>catégoriel</i>).	$\{1, 2, 4\}$
<code>net_arch</code>	Architecture des réseaux MLP acteur et critique (<i>catégoriel</i>).	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

8.3 Résultats de l'optimisation

Cette section présente les résultats obtenus pour chaque algorithme sur les deux variantes de l'environnement LunarLander, en mettant en lumière les configurations optimales identifiées et leur robustesse à travers les différentes seeds.

Le tableau 5 présente les scores obtenus par la meilleure configuration de chaque combinaison algorithme/environnement à l'issue de la phase de tuning. Ces scores correspondent à la récompense moyenne sur 20 épisodes déterministes, obtenue par le meilleur essai parmi les 150 évalués sur 250 000 steps. Ils constituent un indicateur du potentiel maximal de chaque configuration, indépendamment de sa robustesse inter-seeds — qui fait l'objet de la section 8.4.2.

TABLE 5 – Score Optuna des meilleures configurations (récompense moyenne sur 20 épisodes déterministes, meilleur essai parmi 150)

Algorithme	Variante	Score Optuna
DQN	Discret	275.14
PPO	Discret	278.76
PPO	Continu	280.42
SAC	Continu	284.05

Les quatre configurations dépassent largement le seuil de résolution officiel de 200, ce qui confirme l'efficacité du protocole d'optimisation. SAC obtient le meilleur score sur la variante continue, tandis que les deux variantes PPO se situent dans un écart inférieur à 2 points, suggérant une robustesse de l'algorithme aux deux types d'espaces d'action sur cet environnement.

8.3.1 DQN — LunarLander discret

TABLE 6 – Hyperparamètres optimaux — DQN / discret (score Optuna : 275.14)

Hyperparamètre	Valeur optimale
<code>learning_rate</code>	4.73×10^{-4}
<code>buffer_size</code>	10^4
<code>learning_starts</code>	5 000
<code>batch_size</code>	64
<code>gamma</code>	0.99
<code>train_freq</code>	1
<code>target_update_interval</code>	250
<code>exploration_fraction</code>	0.253
<code>exploration_final_eps</code>	0.075
<code>net_arch</code>	<i>medium</i> [256, 256]

La configuration optimale retenue pour DQN présente deux caractéristiques notables. D’une part, la taille du *replay buffer* (10^4) est inférieure aux valeurs typiques de la littérature (10^5 à 10^6 (Mnih et al., 2015)). Cette valeur s’explique vraisemblablement par le budget limité de la phase de tuning (250 000 steps) : avec un petit buffer, les transitions récentes dominent l’échantillonnage, ce qui peut accélérer la convergence à court terme mais reste à confirmer sur les 500 000 steps de l’évaluation finale. D’autre part, `train_freq` = 1 implique une mise à jour du réseau à chaque pas de temps, ce qui, combiné au petit buffer, produit un ratio mise-à-jour/données élevé. Cette configuration agressive a convergé efficacement sur cet environnement, mais introduit potentiellement une variance plus importante dans les gradients.

8.3.2 PPO — LunarLander discret

TABLE 7 – Hyperparamètres optimaux — PPO / discret (score Optuna : 278.76)

Hyperparamètre	Valeur optimale
<code>learning_rate</code>	2.37×10^{-4}
<code>n_steps</code>	2048
<code>batch_size</code>	256
<code>n_epochs</code>	20
<code>gamma</code>	0.995
<code>gae_lambda</code>	0.98
<code>ent_coef</code>	1.55×10^{-2}
<code>clip_range</code>	0.4
<code>max_grad_norm</code>	2.13
<code>vf_coef</code>	0.889
<code>net_arch</code>	<i>large</i> [400, 300]

La configuration PPO discrète se distingue par un `clip_range` de 0.4, valeur maximale de la plage explorée et nettement supérieure à la valeur standard de 0.2 (Schulman; Wolski et al., 2017). Un `clip_range` élevé autorise des mises à jour de politique plus importantes à chaque gradient step, tout en conservant la garantie de stabilité propre à PPO grâce au mécanisme de clipping. Cette valeur, associée à `n_epochs` = 20 et `n_steps` = 2048, indique que PPO effectue $\lfloor 2048/256 \rfloor = 8$ mini-batches⁶ par époque, soit 160 mises à jour du réseau pour chaque rollout de 2048 transitions.

8.3.3 PPO — LunarLanderContinuous

6. 256 étant la taille d'un batch

TABLE 8 – Hyperparamètres optimaux — PPO / continu (score Optuna : 280.42)

Hyperparamètre	Valeur optimale
<code>learning_rate</code>	6.42×10^{-5}
<code>n_steps</code>	2048
<code>batch_size</code>	32
<code>n_epochs</code>	10
<code>gamma</code>	0.995
<code>gae_lambda</code>	0.98
<code>ent_coef</code>	1.27×10^{-4}
<code>clip_range</code>	0.4
<code>max_grad_norm</code>	1.65
<code>vf_coef</code>	0.105
<code>net_arch</code>	<i>large</i> [400, 300]

La comparaison entre les deux variantes PPO révèle une convergence vers des valeurs communes pour plusieurs hyperparamètres structurels (`n_steps` = 2048, `gae_lambda` = 0.98, `clip_range` = 0.4, `net_arch` = *large*), ce qui suggère que ces choix sont robustes à la nature de l'espace d'action sur cet environnement. En revanche, deux hyperparamètres diffèrent significativement entre les deux variantes : le `learning_rate` est quatre fois plus faible pour la variante continue (6.42×10^{-5} contre 2.37×10^{-4}), et l'`ent_coef` est deux ordres de grandeur inférieur (1.27×10^{-4} contre 1.55×10^{-2}). Ces différences indiquent que l'optimisation d'une politique gaussienne en espace continu requiert des mises à jour plus prudentes et une pression d'exploration entropique moindre que dans le cas discret.

8.3.4 SAC — LunarLanderContinuous

TABLE 9 – Hyperparamètres optimaux — SAC / continu (score Optuna : 284.05)

Hyperparamètre	Valeur optimale
<code>learning_rate</code>	9.95×10^{-4}
<code>buffer_size</code>	10^5
<code>learning_starts</code>	1 000
<code>batch_size</code>	512
<code>gamma</code>	0.99
<code>tau</code>	0.05
<code>ent_coef</code>	auto
<code>use_sde</code>	True
<code>target_update_interval</code>	2
<code>net_arch</code>	<i>medium</i> [256, 256]

SAC obtient le meilleur score Optuna (284.05) avec une architecture de réseau *medium* [256, 256], plus petite que celle retenue par PPO (*large* [400, 300]), ce qui illustre l'efficacité par paramètre propre aux algorithmes *off-policy* bénéficiant du *replay buffer*. Deux hyperparamètres méritent d'être soulignés. D'une part, `tau` = 0.05 est la valeur maximale de la plage testée, dix fois supérieure à la valeur standard de 0.005 (Haarnoja ; Zhou ; Abbeel et al., 2018) : cette mise à jour rapide du réseau cible favorise une convergence accélérée sur un environnement de faible dimensionnalité comme LunarLander, où le risque de divergence lié à un réseau cible instable est limité. D'autre part, `use_sde` = True confirme l'intérêt de l'*State Dependent Exploration* pour les espaces d'action continus : le bruit d'exploration structuré en fonction de l'état permet une exploration plus cohérente que le bruit gaussien indépendant de l'état.

8.4 Évaluation finale multi-seeds

8.4.1 Protocole d'évaluation finale

Une fois les hyperparamètres optimaux identifiés pour chaque combinaison algorithme/environnement, une évaluation finale est conduite pour mesurer la robustesse des configurations retenues. Chaque modèle est réentraîné **from scratch** sur **5 seeds indépendantes** ($\{42, 123, 456, 789, 1337\}$) avec 500 000 pas de temps d'entraînement, puis évalué sur 20 épisodes.

Évaluation déterministe. L'évaluation est conduite en mode **déterministe**, ce qui diffère du comportement adopté durant l'entraînement selon l'algorithme :

- **PPO** : pendant l'entraînement, PPO échantillonne ses actions depuis une distribution de probabilités (gaussienne pour le continu, catégorielle pour le discret) afin d'explorer. En mode déterministe, l'action retenue est la *mode* de cette distribution — l'action de probabilité maximale — sans aucun échantillonnage.
- **SAC** : de même, SAC paramétrise une distribution gaussienne et échantillonne lors de l'entraînement pour maximiser l'entropie. En mode déterministe, il retourne la moyenne de cette distribution, ce qui correspond à la politique la plus probable sans bruit d'exploration.
- **DQN** : le mode déterministe consiste à sélectionner systématiquement l'action de Q-valeur maximale (argmax), avec $\varepsilon = 0$. C'est déjà le comportement naturel de DQN lors de l'exploitation ; le mode déterministe ne change donc pas fondamentalement la politique évaluée.

Ce choix d'évaluation déterministe est standard dans la littérature RL : il mesure la politique *apprise* sans bruit d'exploration, offrant une estimation plus fidèle des performances en déploiement réel.

Métriques rapportées. Les métriques suivantes sont calculées sur les 20 épisodes d'évaluation, puis agrégées sur les 5 seeds :

- Récompense moyenne et écart-type inter-seeds,
- Récompense minimale et maximale observées,
- Taux de succès : fraction d'épisodes avec une récompense ≥ 200 (seuil de résolution officiel de LunarLander),
- Nombre de seeds pour lesquelles la récompense moyenne atteint le seuil de résolution.

Cette procédure multi-seeds permet de distinguer les configurations robustes — qui convergent systématiquement — des configurations qui réussissent de manière occasionnelle en raison de la stochasticité de l'entraînement.

8.4.2 Résultats de l'évaluation multi-seeds

Les résultats de l'évaluation multi-seeds sont présentés dans le tableau 10. Chaque configuration optimale est réentraînée sur 5 seeds indépendantes ($\{42, 123, 456, 789, 1337\}$) pendant 500 000 steps, puis évaluée de manière déterministe sur 20 épisodes. Les métriques rapportées sont la récompense moyenne et son écart-type inter-seeds (std échantillon, $n-1$), les récompenses extrêmes observées, le taux de succès (fraction d'épisodes avec une récompense ≥ 200) et le nombre de seeds pour lesquelles la récompense moyenne dépasse le seuil de résolution.

TABLE 10 – Résultats de l'évaluation multi-seeds (500 000 steps, récompense moyenne sur 20 épisodes déterministes, std échantillon)

Algo.	Env.	Mean $\pm$ Std	Min	Max	Succès	Seeds
DQN	Discret	145.19 ± 45.76	101.26	231.89	39%	1/5
PPO	Discret	218.58 ± 50.72	143.95	278.76	77%	3/5
PPO	Continu	208.67 ± 50.17	159.60	278.96	74%	2/5
SAC	Continu	233.44 ± 52.50	132.67	281.01	79%	4/5

SAC et PPO — configurations robustes. SAC obtient la meilleure récompense moyenne (233.44 ± 52.50) et le taux de succès le plus élevé (79%, 4 seeds sur 5 résolues), confirmant la supériorité de l'algorithme à entropie maximale sur la variante continue. PPO affiche des performances solides dans

les deux variantes, avec des récompenses moyennes supérieures au seuil de résolution (218.58 en discret, 208.67 en continu) et des taux de succès proches de 75%. La variante discrète résout légèrement plus de seeds (3/5 contre 2/5), bien que les scores moyens soient proches, ce qui suggère une variance inter-seeds plus favorable en espace discret.

DQN — fragilité de la configuration optimale. DQN présente un écart marqué entre son score Optuna (275.14) et sa récompense moyenne en évaluation finale (145.19), soit une chute de plus de 130 points. Cet écart s'explique vraisemblablement par la nature de la configuration retenue : un *replay buffer* de 10^4 transitions et un `train_freq = 1` constituent une configuration agressive, optimisée pour converger rapidement sur 250 000 steps de tuning, mais qui ne se généralise pas sur les 500 000 steps de l'évaluation finale. Le ratio élevé mise-à-jour/données induit une variance importante dans les gradients, ce qui se traduit par une instabilité inter-seeds (écart-type de 45.76, une seule seed résolue sur 5).

Écarts-types élevés — limite de l'évaluation à 5 seeds. Les écarts-types inter-seeds sont importants pour tous les algorithmes (45–52 points), ce qui reflète la stochasticité inhérente à l'entraînement par renforcement. Avec seulement 5 seeds, ces estimations restent bruitées. Cette phase ayant pour vocation la validation du protocole d'optimisation plutôt qu'une évaluation exhaustive, ce nombre de seeds est néanmoins suffisant pour dégager les tendances principales.

La figure 5 illustre la distribution des temps de convergence inter-seeds. SAC requiert globalement un temps d'entraînement plus important que PPO et DQN, ce qui s'explique par le coût computationnel de ses mises à jour : SAC maintient simultanément deux réseaux Q (double critic) et un acteur, et calcule à chaque step la mise à jour du coefficient d'entropie adaptatif — un coût par update structurellement plus élevé que celui des autres algorithmes, indépendamment de la fréquence des mises à jour. Il est important de mettre en évidence que ces tests ont été réalisés sur un cluster avec des ressources partagées et donc les modèles n'ont pas forcément été entraînés dans des conditions strictement identiques, ce qui peut introduire une variance supplémentaire dans les temps de convergence observés. Cependant, la tendance

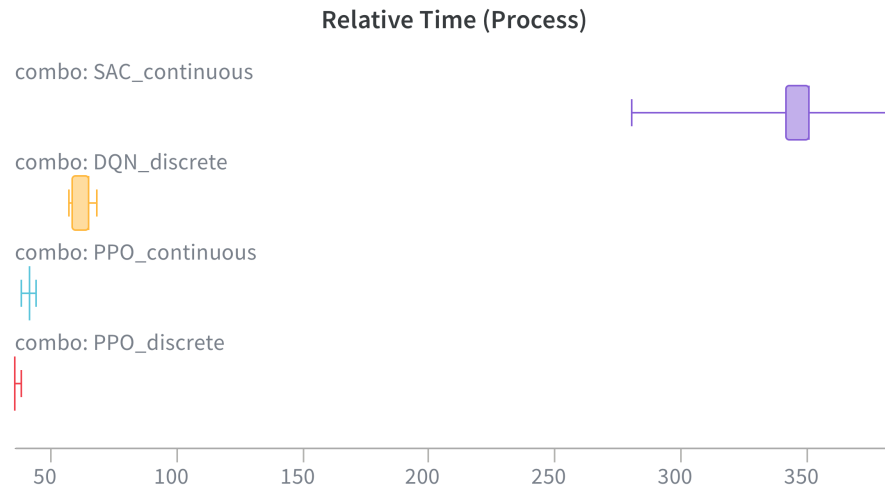


FIGURE 5 – Distribution du temps de convergence (en minutes) pour chaque combinaison algorithme/environnement, sur les 5 seeds indépendantes.

générale reste claire : SAC est plus coûteux en temps d'entraînement que PPO et DQN, ce qui doit être pris en compte dans le choix de l'algorithme en fonction des contraintes de ressources disponibles.

Synthèse. Ces résultats mettent en évidence deux enseignements principaux. D'une part, le score Optuna n'est pas un prédicteur fiable de la robustesse inter-seeds : DQN en est l'illustration la plus nette, avec le deuxième meilleur score de tuning mais les performances finales les plus faibles. D'autre part, SAC et PPO représentent des compromis distincts : SAC offre les meilleures performances sur la variante continue au prix d'un coût computationnel plus élevé, tandis que PPO, applicable aux deux types d'espaces d'action avec des performances stables, constitue un choix plus polyvalent pour des contextes aux ressources limitées.

9 Phase II - Apprentissage sur Rocket League

Pour cette phase 2, le but est de travailler sur un environnement bien plus complexe que LunarLander, nécessitant la maîtrise de mécaniques avancées et une stratégie à long terme. J'ai choisi de travailler sur Rocket League, un jeu de football avec des voitures développé par Psyonix, qui constitue un défi particulièrement riche pour l'apprentissage par renforcement en raison de sa physique réaliste, de la grande variété de situations de jeu possibles et de la nécessité de coordonner des séquences d'actions complexes pour être compétitif.

9.1 Environnement de jeu

Rocket League est un jeu de football avec des voitures dans lequel les joueurs contrôlent des véhicules pour frapper une balle et marquer des buts dans le camp adverse. L'environnement est particulièrement exigeant pour un agent de RL pour plusieurs raisons. D'une part, le jeu tourne nativement à 120 images par seconde, ce qui implique qu'un agent devrait théoriquement prendre 120 décisions par seconde pour interagir avec l'environnement à sa pleine résolution temporelle. D'autre part, les mécaniques de jeu sont nombreuses et interdépendantes : les voitures peuvent récupérer du boost sur le terrain, effectuer des sauts simples ou doubles, réaliser des *flips* (rotations rapides en l'air pouvant conférer une impulsion de vitesse supplémentaire), et coller aux murs ou au plafond de l'arène.

Le jeu peut se pratiquer en mode 1v1, 2v2, 3v3 ou 4v4. Afin de limiter la complexité liée à la coordination entre plusieurs agents, ce travail se concentre exclusivement sur le mode 1v1, où un seul agent affronte un adversaire unique. Ce cadre permet d'isoler les défis liés à l'apprentissage individuel — contrôle du véhicule, gestion du boost, positionnement tactique — sans introduire la dimension supplémentaire de la collaboration multi-agents.

Dans cette phase, ce n'est pas le jeu officiel qui est utilisé, mais un environnement de simulation appelé RocketSim (Section 7.5.1), qui reproduit fidèlement la physique du jeu officiel tout en permettant un entraînement en *headless mode* — sans rendu graphique — ce qui accélère considérablement

les cycles d'expérimentation. RLGym fournit l'interface entre RocketSim et les algorithmes d'apprentissage par renforcement, en exposant les observations, les récompenses et les actions dans un format standardisé compatible avec les bibliothèques de RL modernes (*RLGym* / *RLGym*, [s. d.]).

9.1.1 Espace d'actions

Le vecteur d'action complet dans RLGym est composé de huit dimensions, décrites dans le tableau 11. Cinq d'entre elles sont continues (throttle, steer, pitch, yaw, roll) et trois sont discrètes (jump, boost, handbrake).

Action	Plage	Type	Description
Throttle	[-1,1]	Continu	Accélération/décélération du véhicule
Steer	[-1,1]	Continu	Direction de braquage au sol
Pitch	[-1,1]	Continu	Rotation verticale avant/arrière (en l'air)
Yaw	[-1,1]	Continu	Rotation horizontale (en l'air)
Roll	[-1,1]	Continu	Rotation latérale (en l'air)
Jump	0,1	Discret	Saut simple ou double
Boost	0,1	Discret	Utilisation du boost
Handbrake	0,1	Discret	Frein à main (déraper)

TABLE 11 – Espace d'actions dans RLGym (*RLGym* / *RLGym*, [s. d.])

L'espace d'actions brut, en considérant toutes les combinaisons discrétisées possibles de ces huit dimensions, atteint 1944 combinaisons. Entraîner un agent sur un espace aussi vaste ralentirait considérablement la convergence, car une grande partie de ces combinaisons est redondante ou mécaniquement incohérente (par exemple, utiliser le boost sans accélérer, ou effectuer un yaw et un jump simultanément en l'air).

Pour remédier à cela, une *lookup table* d'actions est utilisée via le parser `LookupTableAction` de RLGym (*RLGym* / *RLGym*, [s. d.]), qui réduit l'espace d'actions à **90 combinaisons discrètes** en ne conservant que les ac-

tions mécaniquement pertinentes. Ces 90 actions se décomposent en deux blocs :

Actions au sol (18 actions). Elles couvrent toutes les combinaisons utiles de $\text{throttle} \in \{-1, 0, 1\}$, $\text{steer} \in \{-1, 0, 1\}$, $\text{boost} \in \{0, 1\}$ et $\text{handbrake} \in \{0, 1\}$, sous la contrainte que le boost n'est activé que si le throttle est au maximum ($\text{boost} = 1 \Rightarrow \text{throttle} = 1$), ce qui correspond au comportement physique réel du jeu :

$$|\mathcal{A}_{\text{sol}}| = |\text{throttle}| \times |\text{steer}| \times |\text{boost}| \times |\text{handbrake}| - \text{cas invalides} = 18 \quad (22)$$

Actions aériennes (72 actions). Elles couvrent les combinaisons de pitch, yaw, roll, jump et boost lorsque le véhicule est en l'air. Les doublons avec les actions au sol sont supprimés, ainsi que les combinaisons yaw+jump (remplacées par roll pour les flips horizontaux) :

$$|\mathcal{A}_{\text{air}}| = 72 \quad (23)$$

Au total : $|\mathcal{A}| = 18 + 72 = 90$ actions discrètes.

Ce choix d'un espace discret réduit présente deux avantages majeurs. D'une part, il est directement compatible avec PPO, dont la couche de sortie applique une distribution catégorielle sur les actions discrètes (Section 5.8.2). D'autre part, il réduit significativement la complexité de l'exploration, ce qui favorise une convergence plus rapide — particulièrement important dans un environnement qui nécessite plusieurs milliards de pas d'entraînement pour atteindre un niveau de jeu compétitif.

SAC ne sera pas exploré dans cette phase pour deux raisons complémentaires. Premièrement, comme établi en Section 5.8.3, SAC est conçu pour les espaces d'actions continus et repose sur la maximisation de l'entropie d'une distribution gaussienne sur les actions. Il n'est donc pas directement applicable à un espace d'actions discret tel que celui retenu ici. Deuxièmement, les résultats de la Phase I ont mis en évidence que SAC présente un coût computationnel structurellement plus élevé que PPO, indépendamment de la nature de l'espace d'action : la maintenance simultanée de deux réseaux Q, de deux

réseaux cibles et du mécanisme d’ajustement adaptatif du coefficient d’entropie implique un coût par mise à jour significativement supérieur. Dans un environnement nécessitant plusieurs milliards de pas d’entraînement, ce surcoût devient prohibitif au regard des ressources de calcul disponibles. PPO constitue donc le choix naturel pour cette phase, à la fois pour sa compatibilité native avec les espaces discrets et pour son efficacité computationnelle démontrée.

Répétition d’actions (*action repeat*). À la fréquence native de 120 ticks par seconde, un agent devrait théoriquement prendre 120 décisions distinctes par seconde. En pratique, cette granularité temporelle est contre-productive pour l’apprentissage : elle génère un signal de récompense extrêmement bruité, rend le problème d’assignation de crédit (*credit assignment*) particulièrement difficile — il devient ardu pour l’agent de déterminer quelle action parmi les 120 a réellement contribué à une récompense donnée — et s’éloigne considérablement de la fréquence décisionnelle humaine.

Pour remédier à cela, un mécanisme de répétition d’actions (**RepeatAction**) est appliqué avec un facteur de 8 ticks, ramenant la fréquence décisionnelle effective de l’agent à $120/8=15$ Hz. Chaque action sélectionnée par la politique est ainsi maintenue pendant 8 ticks consécutifs avant qu’une nouvelle décision soit prise. Ce principe, connu sous le nom de *frame skipping* dans la littérature du Deep RL appliqué aux jeux vidéo, a été introduit et validé empiriquement par Mnih et al. (2015) dans le cadre d’Atari, où un facteur de répétition de 4 frames est appliqué. Il est depuis devenu une pratique standard dans les environnements à haute fréquence de simulation.

9.1.2 Espace d’observation

L’espace d’observation fournit à l’agent un vecteur d’état normalisé décrivant la situation de jeu à chaque pas de temps. Toutes les valeurs sont normalisées pour être contenues dans l’intervalle $[-1, 1]$, ce qui est une pratique standard en Deep RL pour stabiliser l’apprentissage et éviter que certaines dimensions ne dominent numériquement les autres (Sutton et al., 2015).

Le vecteur d’observation décrit chaque entité de jeu — la voiture de l’agent,

la voiture adverse et la balle — à travers les informations suivantes :

- **Position** (x, y, z) normalisée par les dimensions de l'arène.
- **Orientation** du véhicule dans l'espace 3D, encodée sous forme des vecteurs *forward*, *up* et *right*.
- **Vitesse linéaire** (v_x, v_y, v_z) normalisée par la vitesse maximale atteignable.
- **Vitesse angulaire** $(\omega_x, \omega_y, \omega_z)$ normalisée par la vitesse angulaire maximale.
- **Quantité de boost** disponible, normalisée sur $[0, 1]$.
- **Variables d'état booléennes** : contact avec le sol, disponibilité du saut, état de démolition du véhicule.

Ce vecteur est de taille fixe, ce qui est cohérent avec le mode 1v1 retenu : le nombre d'entités étant constant à chaque pas de temps, aucun rembourrage dynamique n'est nécessaire.

Cet espace d'observation est dit *omniscient* : l'agent dispose en permanence d'informations complètes sur l'ensemble de l'environnement, ce qui le place dans des conditions bien plus favorables qu'un joueur humain. En particulier, la position et la vitesse exactes de la balle, ainsi que le positionnement et le niveau de boost de l'adversaire, sont accessibles à tout moment — des informations qu'un joueur humain ne peut qu'estimer visuellement et imparfaitement.

10 Discussion transversale

11 Conclusion

Références

AKIBA, Takuya ; SANO, Shotaro ; YANASE, Toshihiko ; OHTA, Takeru ; KOYAMA, Masanori, 2019. *Optuna : A Next-generation Hyperparam-*

- ter Optimization Framework* [en ligne]. 2019-07-25. [visité le 2026-05-04]. Disp. à l'adr. DOI : 10.48550/arXiv.1907.10902.
- FUJIMOTO, Scott ; HOOFF, Herke van ; MEGER, David, 2018. *Addressing Function Approximation Error in Actor-Critic Methods* [en ligne]. 2018-10-22. [visité le 2026-05-13]. Disp. à l'adr. DOI : 10.48550/arXiv.1802.09477.
- Gymnasium Documentation*, 2025 [en ligne]. [visité le 2026-04-21]. Disp. à l'adr.: <https://gymnasium.farama.org/index.html>.
- Gymnasium Documentation - Lunar Lander*, [n.d.] [en ligne]. [visité le 2026-05-04]. Disp. à l'adr.: https://gymnasium.farama.org/environments/box2d/lunar_lander.html.
- HAARNOJA, Tuomas ; ZHOU, Aurick ; ABBEEL, Pieter ; LEVINE, Sergey, 2018. *Soft Actor-Critic : Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor* [en ligne]. 2018-08-08. [visité le 2026-03-31]. Disp. à l'adr. DOI : 10.48550/arXiv.1801.01290.
- HAARNOJA, Tuomas ; ZHOU, Aurick ; HARTIKAINEN, Kristian ; TUCKER, George ; HA, Sehoon ; TAN, Jie ; KUMAR, Vikash ; ZHU, Henry ; GUPTA, Abhishek ; ABBEEL, Pieter ; LEVINE, Sergey, 2019. *Soft Actor-Critic Algorithms and Applications* [en ligne]. 2019-01-29. [visité le 2026-05-11]. Disp. à l'adr. DOI : 10.48550/arXiv.1812.05905.
- IFTIKHAR, Arta ; GHAZANFAR, Mustansar Ali ; AYUB, Mubbashir ; ALI ALAHMARI, Saad ; QAZI, Nadeem ; WALL, Julie, 2024. A Reinforcement Learning Recommender System Using Bi-Clustering and Markov Decision Process. *Expert Systems with Applications* [en ligne]. T. 237, p. 121541 [visité le 2026-02-23]. ISSN 0957-4174. Disp. à l'adr. DOI : 10.1016/j.eswa.2023.121541.
- LILLICRAP, Timothy P. ; HUNT, Jonathan J. ; PRITZEL, Alexander ; HEESS, Nicolas ; EREZ, Tom ; TASSA, Yuval ; SILVER, David ; WIERSTRA, Daan, 2015. *Continuous Control with Deep Reinforcement Learning* [en ligne]. arXiv.org, 2015-09-09. [visité le 2026-04-13]. Disp. à l'adr.: <https://arxiv.org/abs/1509.02971v6>.

- MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David; RUSU, Andrei A.; VENESS, Joel; BELLEMARE, Marc G.; GRAVES, Alex; RIEDMILLER, Martin; FIDJELAND, Andreas K.; OSTROVSKI, Georg; PETERSEN, Stig; BEATTIE, Charles; SADIK, Amir; ANTONOGLOU, Ioannis; KING, Helen; KUMARAN, Dharshan; WIERSTRA, Daan; LEGG, Shane; HASSABIS, Demis, 2015. Human-Level Control through Deep Reinforcement Learning. *Nature* [en ligne]. Vol. 518, no. 7540, p. 529–533 [visit  le 2026-03-31]. ISSN 1476-4687. Disp.   l’adr. DOI: 10.1038/nature14236.
- OpenAI Five*, 2018 [en ligne]. 2018-06-25. [visit  le 2026-02-23]. Disp.   l’adr.: <https://openai.com/index/openai-five/>.
- RAFFIN, Antonin ; GALLOU DEC, Quentin ; DORMANN, Noah ; GLEAVE, Adam ; ANSSI ; PASQUALI, Alex ; ROCAMONDE, Juan ; ERNESTUS, M. ; HELM, Patrick ; CORENTIN ; SIMONINI, Thomas ; ROHRER, Tobias ; TIO, Sidney ; TANGRI, Rohan ; TOWERS, Mark ; D RR, Tom ; UNEXPLOREDTEST ; JALLET, Wilson ; WANG, Steven H. ; TOYER, Sam ; GAVRILESCU, Roland ; SCHEIKL, Paul Maria ; KOTHARI, Parth ; KACHAIEV, Oleksii ; KLAIBER, Megan ; RAML, Bernhard ; SCHINDLBECK, Chris ; HUANG, Costa ; KERR, Dominic, 2026. *DLR-RM/Stable-Baselines3 : V2.8.0 : Dropped Python 3.9, Added Python 3.13 Support, MaskablePPO Bug Fix, Default Hyperparams for Unlisted Env in the RL Zoo, Markdown Doc* [en ligne]. Zenodo. Version v2.8.0 [visit  le 2026-04-21]. Disp.   l’adr. DOI : 10.5281/ZENODO.8123988.
- RAFFIN, Antonin ; KOBER, Jens ; STULP, Freek, 2021. *Smooth Exploration for Robotic Reinforcement Learning* [en ligne]. 2021-06-20. Version 2 [visit  le 2026-05-13]. Disp.   l’adr. DOI : 10.48550/arXiv.2005.05719.
- RLGym / RLGym*, [n.d.] [en ligne]. [visit  le 2026-04-21]. Disp.   l’adr.: <https://rlgym.org/>.
- SCHULMAN, John ; MORITZ, Philipp ; LEVINE, Sergey ; JORDAN, Michael ; ABBEEL, Pieter, 2018. *High-Dimensional Continuous Control Using Generalized Advantage Estimation* [en ligne]. 2018-10-20. [visit  le 2026-04-13]. Disp.   l’adr. DOI : 10.48550/arXiv.1506.02438.
- SCHULMAN, John ; WOLSKI, Filip ; DHARIWAL, Prafulla ; RADFORD, Alec ; KLIMOV, Oleg, 2017. *Proximal Policy Optimization Algorithms* [en ligne]. arXiv.org, 2017-07-20. [visit  le 2026-03-31]. Disp.   l’adr.: <https://arxiv.org/abs/1707.06347v2>.

- SILVER, David, 2015. *Lectures on Reinforcement Learning*.
- SUTTON, Richard S; BARTO, Andrew G, 2015. Reinforcement Learning: An Introduction.
- TANG, Chen ; ABBATEMATTEO, Ben ; HU, Jiaheng ; CHANDRA, Rohan ; MARTÍN-MARTÍN, Roberto ; STONE, Peter, 2024. *Deep Reinforcement Learning for Robotics : A Survey of Real-World Successes* [en ligne]. 2024-09-16. [visité le 2026-02-23]. Disp. à l'adr. DOI : 10.48550/arXiv.2408.03539.
- Weights & Biases, [n.d.]. *Weights & Biases: The AI Developer Platform* [en ligne]. Weights & Biases. [visité le 2026-04-21]. Disp. à l'adr.: <https://wandb.ai/site/>.
- WILLIAMS, Ronald J., 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Mach Learn* [en ligne]. Vol. 8, no. 3, p. 229–256 [visité le 2026-04-13]. ISSN 1573-0565. Disp. à l'adr. DOI: 10.1007/BF00992696.
- ZAFFAGNI, Marc, 2022. *Définition / AlphaGo : qu'est-ce que c'est ? / Futura tech* [en ligne]. Futura, 2022-12-01. [visité le 2026-02-23]. Disp. à l'adr. : <https://www.futura-sciences.com/tech/definitions/intelligence-artificielle-alphago-16467/>.
- ZEALANL, 2022. *ZealanL/RocketSim* [en ligne]. [visité le 2026-04-22]. Disp. à l'adr. : <https://github.com/ZealanL/RocketSim>.
- ZEALANL, 2024. *ZealanL/RocketSimVis* [en ligne]. [visité le 2026-06-11]. Disp. à l'adr. : <https://github.com/ZealanL/RocketSimVis>.